

Assignment 2: Adversarial Search (Oxono)

Douaa El Mandili
Edouard Lacheron
G090

1 Introduction

Dans le cadre de ce projet, nous avons développé un agent pour le jeu Oxono en utilisant des techniques de recherche adversariale. L'objectif est de concevoir un agent capable de jouer de manière compétitive face à différents types d'adversaires.

Nous avons choisi d'utiliser l'algorithme Alpha-Beta, une amélioration du Minimax, permettant d'explorer efficacement l'arbre de jeu tout en réduisant le nombre de positions évaluées. Toutefois, les performances de cet algorithme dépendent fortement de la qualité de la fonction d'évaluation utilisée pour estimer les positions non terminales.

Au cours du développement, plusieurs versions de l'agent ont été conçues et testées. Nous avons progressivement amélioré la fonction d'évaluation et introduit différentes heuristiques, notamment pour mieux gérer les situations critiques telles que les victoires immédiates ou les blocages nécessaires.

Enfin, nous avons réalisé plusieurs expériences afin de comparer les performances des différentes versions de notre agent et d'analyser l'impact des améliorations apportées. La version finale retenue (V6) correspond à la meilleure combinaison entre efficacité et stabilité.

2 Design de l'agent

2.1 Algorithme de décision

Notre agent repose sur l'algorithme Alpha-Beta, une amélioration du Minimax classique permettant de réduire le nombre de nœuds explorés dans l'arbre de jeu.

L'algorithme est implémenté à l'aide de deux fonctions récursives : `max_value` et `min_value`. La fonction `max_value` cherche à maximiser le score pour

notre agent, tandis que `min_value` modélise le comportement de l'adversaire en minimisant ce score.

À chaque appel, les actions possibles sont générées via `Game.actions(state)`. Pour chaque action, une copie de l'état est créée (`state.copy()`) afin d'éviter toute modification de l'état courant, puis l'action est appliquée avec `Game.apply`.

L'exploration se poursuit récursivement jusqu'à atteindre soit un état terminal, soit une profondeur maximale. Dans le cas d'un état terminal, la fonction `Game.utility` est utilisée pour retourner un score élevé ou faible selon le résultat. Lorsque la profondeur limite est atteinte, une fonction d'évaluation est utilisée pour estimer la qualité de la position.

Un mécanisme d'élagage Alpha-Beta est également utilisé afin de ne pas explorer certaines branches inutiles. Cela permet de réduire considérablement le temps de calcul tout en conservant une bonne qualité de décision. Enfin, dans la version finale (V6), une heuristique supplémentaire est introduite afin de mieux prendre en compte les situations critiques.

2.2 Gestion du temps et du premier coup

Une gestion du temps est intégrée afin de respecter les contraintes imposées. La profondeur de recherche est adaptative : elle est fixée à 4 lorsque le temps restant est supérieur à 20 secondes, et réduite à 3 sinon. Une condition d'arrêt est également utilisée afin d'interrompre la recherche si le temps d'exécution dépasse une fraction du temps disponible.

Lors du premier coup, l'agent sélectionne une action aléatoire parmi les actions possibles. Ce choix permet de réduire le temps de calcul dans des positions initiales souvent symétriques.

2.3 Fonction d'évaluation

La fonction d'évaluation est utilisée lorsque la profondeur maximale est atteinte. Les états terminaux sont traités en priorité avec des scores extrêmes, en utilisant la fonction `Game.utility`.

L'évaluation repose sur une analyse de fenêtres de taille 4 sur le plateau. Pour chaque fenêtre, le nombre de pions appartenant au joueur et à l'adversaire est compté. Les alignements du joueur sont valorisés, tandis que ceux de l'adversaire sont pénalisés.

Dans notre implémentation, les alignements de taille 3 sont fortement pondérés, car ils représentent des situations proches de la victoire. Cela permet d'orienter efficacement la recherche vers des positions prometteuses tout en tenant compte des menaces adverses. Enfin, une heuristique supplémentaire

permet d'identifier les coups menant à une victoire au tour suivant en simulant les actions possibles. Contrairement à une détection directe, cette information est intégrée dans la fonction d'évaluation sous forme de score, ce qui influence la décision sans court-circuiter l'algorithme Alpha-Beta.

2.4 Optimisations

L'élagage Alpha-Beta permet de réduire significativement le nombre de nœuds explorés en évitant l'exploration de branches inutiles.

Par ailleurs, l'utilisation de copies d'état (`state.copy()`) permet d'explorer les différentes actions sans modifier l'état courant.

Enfin, l'adaptation de la profondeur et la gestion du temps d'exécution contribuent à maintenir un bon compromis entre qualité des décisions et contraintes temporelles.

3 Évolution de l'agent

3.1 Versions initiales (V1 et V2)

Les premières versions implémentent directement Alpha-Beta avec une profondeur fixe et une fonction d'évaluation simple.

La fonction `evaluate_color_window` utilise une pondération exponentielle (par exemple 2×10^k) en fonction du nombre de pions alignés, ce qui donne des scores très instables.

Ces versions sont testées contre un agent aléatoire et permettent de gagner de manière fiable, mais elles restent peu performantes face à des agents structurés.

3.2 Version intermédiaire (V3)

La version V3 introduit deux changements principaux :

- une profondeur adaptative en fonction du temps restant ;
- une nouvelle fonction d'évaluation basée sur des scores fixes (par exemple 500 pour 3 alignés).

Contrairement à V1, la fonction d'évaluation distingue explicitement les cas (1, 2, 3 ou 4 alignés) avec des poids fixes, ce qui stabilise les décisions.

Cette version améliore significativement les performances et atteint environ 60% de victoire contre l'agent de référence.

3.3 Versions V4 et V5

Les versions V4 et V5 correspondent à des tests sur les pondérations et les pénalités dans la fonction d'évaluation.

Ces modifications ont souvent conduit à un déséquilibre, notamment en défense, ce qui se traduit par des défaites fréquentes lorsque l'agent joue en second.

3.4 Version finale (V6)

La version V6 introduit une amélioration de la gestion des situations critiques via la fonction d'évaluation.

Une heuristique supplémentaire permet d'identifier les coups menant à une victoire au tour suivant en simulant les actions possibles. Cette information est intégrée sous forme de score, ce qui permet d'orienter l'agent vers ces positions favorables.

De plus, les menaces adverses sont également prises en compte, ce qui améliore partiellement le comportement défensif de l'agent.

Contrairement à une approche basée sur une détection directe des coups critiques, cette stratégie reste intégrée dans l'algorithme Alpha-Beta, ce qui permet de conserver une cohérence globale dans la prise de décision.

Cette version obtient environ $\sim 60\text{--}70\%$ de victoire contre l'agent de référence, avec un comportement plus stable que les versions précédentes.

4 Expériences

4.1 Résultats

Version	Ordre de jeu	Performance typique	Observation
V1–V2	Premier / Second	Élevée vs random	Fonctionnel mais limité
V3	Premier	30–60%	Bon comportement offensif
V3	Second	10–30%	Capacités défensives limitées
V4–V5	Premier / Second	10–30%	Modifications inefficaces
V6	Premier / Second	$\sim 60\text{--}70\%$	Comportement plus équilibré

Ces résultats correspondent aux évaluations réalisées sur INGINIOUS.

Lors des tests locaux, nous avons observé un comportement différent. Les premières versions de l'agent (V1 à V3) gagnaient de manière systématique

lorsqu’elles jouaient en attaque, mais échouaient souvent en défense, en particulier lorsqu’elles jouaient en second.

Les versions intermédiaires ont tenté d’améliorer cet aspect, sans succès. En revanche, la version V6 permet d’obtenir un comportement plus équilibré. Lors de tests locaux, V6 parvient à battre V3 dans plusieurs configurations, ce qui montre une amélioration de la robustesse globale de l’agent.

Sur INGINious, l’agent atteint un taux de victoire global d’environ $\sim 60\text{--}70\%$ contre l’agent de référence, avec des performances similaires en tant que premier et second joueur. Cela indique que l’agent est relativement équilibré et moins dépendant de l’ordre de jeu.

Ces observations montrent que la combinaison de l’algorithme Alpha-Beta et de la fonction d’évaluation permet d’obtenir un comportement globalement stable, tout en restant sensible aux situations défensives.

5 Discussion et Conclusion

Les résultats montrent que notre agent atteint un comportement globalement stable, avec des performances similaires en tant que premier et second joueur.

Les améliorations apportées dans la version V6 permettent de mieux gérer les situations critiques, en particulier en défense, tout en conservant une stratégie offensive efficace.

Les différentes expérimentations ont également montré que l’ajout d’heuristiques ne garantit pas nécessairement une amélioration des performances. Certaines modifications ont dégradé le comportement global en introduisant des décisions trop locales.

Ainsi, il est essentiel de maintenir un équilibre entre la fonction d’évaluation et l’algorithme Alpha-Beta afin de préserver la cohérence de la prise de décision.

Dans ce projet, nous avons développé un agent basé sur Alpha-Beta et une fonction d’évaluation heuristique. Après plusieurs itérations, la version V6 s’est révélée être la plus efficace, offrant un bon compromis entre performance et robustesse.

Ce travail met en évidence qu’un agent relativement simple mais bien calibré peut être plus performant qu’un agent plus complexe dont les heuristiques sont mal équilibrées.